Homework Assignment 4

Solution Manual

Question 1 Remainders of squares

(a) The only possible remainders are 0 and 1.

The four computations give

r	0	1	2	3
r^2	0	1	4	9
$r^2 \bmod 4$	0	1	0	1

For every integer n, choose its remainder $r \in \{0, 1, 2, 3\}$ modulo 4. Since $n \equiv r \pmod 4$, the multiplication rule for congruences gives $n^2 \equiv r^2 \pmod 4$. The table therefore covers every integer, including negative integers, and proves that n^2 has remainder 0 or 1 modulo 4.

(b) A sum of two squares cannot have remainder 3.

Each square has remainder 0 or 1. Adding the possible remainders gives

$$0 + 0 = 0, \qquad 0 + 1 = 1, \qquad 1 + 0 = 1, \qquad 1 + 1 = 2.$$

Thus the possible remainders of $x^2 + y^2$ modulo 4 are exactly $0, 1, 2$. All three occur, as shown by $0^2 + 0^2$, $0^2 + 1^2$, and $1^2 + 1^2$.

If $x^2 + y^2 = 4k + 3$, reducing modulo 4 would give remainder 3 for the sum of two squares, contradicting the list. Hence this equality is impossible for integers x, y, k. In particular, $31 = 4 \cdot 7 + 3$, so $x^2 + y^2 = 31$ has no integer solutions. No search over x and y is needed.

(c) Passing the test is not sufficient.

The claim is false. The number 6 has remainder 2 modulo 4, so it passes the remainder test. However, if $x^2 + y^2 = 6$, both squares must lie in $\{0, 1, 4\}$. These are all the nonnegative integer squares at most 6; negative choices of x or y do not produce additional square values.

If $x^2 = 0$, then y^2 would have to be 6; if $x^2 = 1$, then y^2 would have to be 5; and if $x^2 = 4$, then y^2 would have to be 2. None is a square. Thus 6 cannot be written as a sum of two integer squares.

Having remainder 0, 1, or 2 modulo 4 is a necessary condition for being a sum of two squares, but it is not sufficient. The test rules out some impossible values; it does not establish that every value it permits has a representation.

Question 2 Different coefficients in Bézout's identity

(a) One gcd, different coefficient pairs.

Both proposed pairs give the same value:

$$12(2) + 20(-1) = 24 - 20 = 4, \qquad 12(-3) + 20(2) = -36 + 40 = 4.$$

The gcd is a uniquely determined positive integer, here 4. Bézout's identity represents that integer as a linear combination. Different coefficient pairs can give the same value, so the uniqueness of the gcd does not imply uniqueness of its coefficients.

(b) Changing the coefficients without changing the sum.

Increasing x by 5 increases $12x$ by 60. To compensate, $20y$ must decrease by 60, so y must decrease by 3. Thus $(x, y) \mapsto (x + 5, y - 3)$ preserves the sum. Applying this change twice, starting from $(2, -1)$, gives

$$(2, -1) \longrightarrow (7, -4) \longrightarrow (12, -7).$$

These two new pairs satisfy

$$12(7) + 20(-4) = 84 - 80 = 4, \qquad 12(12) + 20(-7) = 144 - 140 = 4.$$

Other valid pairs are also acceptable.

(c) Infinitely many representations.

For every integer t,

$$\begin{aligned} 12(2 + 5t) + 20(-1 - 3t) &= 24 + 60t - 20 - 60t \\ &= 4. \end{aligned}$$

Different integer values of t give different values of $x = 2+5t$, so they give distinct coefficient pairs. Since there are infinitely many integers t, this constructs infinitely many such pairs. In particular, $t = 0$ and $t = -1$ give the pairs from part (a).

(d) A negative coefficient is necessary here.

If $x, y \geq 0$ and at least one is positive, then $12x+20y \geq 12$, so the sum cannot equal 4. If both are zero, the sum is zero. Thus both coefficients cannot be nonnegative. They also cannot both be nonpositive, since their sum would then be nonpositive. Therefore every solution in this example has one positive coefficient and one negative coefficient. This conclusion concerns these particular inputs; it is not a claim about all Bézout identities.

Question 3 Finding a maximum in two different ways

(a) Correctness.

For `MaxOne`, induct on the length $n \geq 1$. At $n = 1$, the only entry is the maximum. For $n \geq 2$, the induction hypothesis says that the recursive call returns the largest entry among the first $n - 1$ entries. Comparing it with the last entry therefore returns the largest entry of the whole array.

For `MaxTwo`, write $n = 2^k$ and induct on $k \geq 0$. At $k = 0$, the array has one entry, which is returned. For $k \geq 1$, each half has length 2^{k-1}. By induction, the two recursive calls return the maxima of their respective halves. Every array entry belongs to one of these halves, so the larger of their two maxima is the maximum of the whole array. Both proofs allow ties: comparing equal maxima still returns the correct maximum value.

(b) Comparison counts.

A base call performs no comparisons. Each nonbase call compares two values once, in addition to the work of its recursive calls:

$$C_1(1) = 0, \quad C_1(n) = C_1(n-1) + 1,$$
$$C_2(1) = 0, \quad C_2(n) = 2C_2(n/2) + 1.$$

The first recurrence holds for all integers $n \geq 2$; the second holds for powers of 2 with $n \geq 2$. Expanding the first until the argument is 1 gives

$$C_1(n) = C_1(1) + (n-1) = \boxed{n-1}.$$

For the second, let $n = 2^k$. At depth $j < k$ there are 2^j nonbase calls, each contributing one comparison. Hence

$$C_2(n) = \sum_{j=0}^{k-1} 2^j = 2^k - 1 = \boxed{n-1}.$$

For $n = 1$ the sum is empty and equals zero. At $n = 8$, each algorithm makes exactly seven comparisons. Both counts grow linearly.

(c) Different depths, equal total work.

The `MaxOne` call tree is a single path with input lengths $n, n-1, \ldots, 1$. It has depth $n - 1$ edges and $n - 1$ nonbase calls. Each of those calls makes one comparison.

The `MaxTwo` tree has depth $k = \log_2 n$ edges. However, it branches into two calls at every nonbase node. It has $1 + 2 + \cdots + 2^{k-1} = n - 1$ nonbase nodes, each making one comparison. For $n = 8$, the depths are seven and three, while both comparison counts are seven. At $n = 1$, both depths are zero.

Halving the input bounds the length of a path. It does not bound the total number of calls by that path length. The student's argument overlooks the growing number of calls on each level. In the specified comparison model, the balanced recursion reduces depth but does not reduce the total work.

Question 4 Solving linear recurrences

(a) Two distinct roots.

The characteristic equation is

$$r^2 - 6r + 8 = (r-2)(r-4) = 0.$$

The roots are 2 and 4, so write

$$a_n = A2^n + B4^n.$$

The initial values give

$$A + B = 3, \qquad 2A + 4B = 8.$$

Subtracting twice the first equation from the second gives $2B = 2$, so $B = 1$ and $A = 2$. Thus

$$\boxed{a_n = 2 \cdot 2^n + 4^n.}$$

At $n = 2$, the recurrence gives $a_2 = 6(8) - 8(3) = 24$. The formula gives $2 \cdot 2^2 + 4^2 = 8 + 16 = 24$, as required.

(b) A repeated root.

The characteristic equation is

$$r^2 - 6r + 9 = (r-3)^2 = 0.$$

The root 3 occurs twice. The repeated-root rule gives

$$b_n = (A + Bn)3^n.$$

From $b_0 = 1$ we get $A = 1$. From $b_1 = 6$ we get $3(A + B) = 6$, so $B = 1$. Hence

$$\boxed{b_n = (n+1)3^n.}$$

At $n = 2$, the recurrence gives $b_2 = 6(6) - 9(1) = 27$. The formula gives $(2+1)3^2 = 27$, as required.

Question 5 Analyzing recurrences

(a) An arithmetic sum.

After j expansions,

$$T_1(n) = T_1(n-j) + \sum_{i=0}^{j-1} \big(2(n-i)-1\big).$$

The base case is reached when $n - j = 1$, so $j = n - 1$. Thus

$$T_1(n) = \sum_{r=2}^{n} (2r-1) = \left(\sum_{r=1}^{n} (2r-1)\right) - 1 = \boxed{n^2 - 1}.$$

The empty sum gives $T_1(1) = 0$, as required. Its growth is quadratic: $T_1(n) = O(n^2)$.

(b) A geometric sum along one path.

Write $n = 2^k$. After j expansions,

$$T_2(n) = T_2(n/2^j) + n \sum_{i=0}^{j-1} 2^{-i}.$$

We stop at $j = k$, when $n/2^k = 1$. Therefore

$$T_2(n) = n \frac{1 - 2^{-k}}{1 - 1/2} = \boxed{2n - 2}.$$

The formula includes $n = 1$, where $k = 0$ and the sum is empty. There is only one call per level, and its local work halves at each step. The total is linear: $T_2(n) = O(n)$.

(c) Constant total work on each level.

At depth j, there are 2^j calls, each on an input of size $n/2^j$. Their total local work is

$$2^j \cdot \frac{n}{2^j} = n.$$

For $n = 2^k$, the internal levels are $j = 0, \ldots, k-1$. The leaves at depth k contribute $T_3(1) = 0$, so

$$T_3(n) = kn = \boxed{n \log_2 n} = O(n \log n).$$

At $n = 1$ this is zero. Both parts (b) and (c) have depth k, but their widths differ. In (b) there is one shrinking subproblem per level; in (c) the number of subproblems doubles, keeping the combined local work equal to n at every internal level.

Question 5 Analyzing recurrences (continued)

(d) Guessing and checking a linear bound.

The recursive input sizes add to at most $n/3 + n/2 = 5n/6$. This suggests a linear bound. We prove $T_4(n) \leq 6n$ for every integer $n \geq 0$ by strong induction.

The base cases give $T_4(0) = 0 \leq 0$ and $T_4(1) = 0 \leq 6$. For $n \geq 2$, both recursive arguments are nonnegative integers strictly smaller than n. Assuming the bound for all smaller arguments gives

$$\begin{aligned} T_4(n) &\leq 6\lfloor n/3 \rfloor + 6\lfloor n/2 \rfloor + n \\ &\leq 6(n/3) + 6(n/2) + n = 6n. \end{aligned}$$

Thus $T_4(n) = O(n)$. More generally, substituting Cn would require

$$\frac{5}{6}C + 1 \leq C \quad \Longleftrightarrow \quad C \geq 6.$$

The recursive calls use at most five-sixths of the proposed bound; the remaining sixth pays for the additional n units of work. Rounding down can only decrease the upper bound used in this proof.

(e) Guessing and checking a quadratic bound.

Since $T_5(n) \geq n^2$ for $n \geq 2$, a linear bound cannot work for all powers of 2. Try $T_5(n) \leq Cn^2$ instead. Substituting this guess gives

$$3C(n/2)^2 + n^2 = \left(\frac{3}{4}C + 1\right) n^2.$$

To make this at most Cn^2, it is enough to choose $C \geq 4$. We prove $T_5(2^k) \leq 4(2^k)^2$ by induction on $k \geq 0$.

For $k = 0$, the base case is $T_5(1) = 1 \leq 4$. For $k \geq 1$, set $n = 2^k$ and apply the induction hypothesis to $n/2 = 2^{k-1}$:

$$T_5(n) = 3T_5(n/2) + n^2 \leq 3 \cdot 4(n/2)^2 + n^2 = 4n^2.$$

Therefore $T_5(n) = O(n^2)$ on the specified domain. Although there are three recursive calls, their squared input sizes add to only $3n^2/4$, leaving room for the local work in the quadratic bound.

Question 6 Proving the Chinese Remainder Theorem

(a) Verifying the given solution.

The identity $au + bv = 1$ gives $bv \equiv 1 \pmod{a}$ and $au \equiv 1 \pmod{b}$. Also $au \equiv 0 \pmod{a}$ and $bv \equiv 0 \pmod{b}$. Therefore

$$x_0 = rbv + sau \equiv r \cdot 1 + s \cdot 0 = r \pmod{a},$$
$$x_0 = rbv + sau \equiv r \cdot 0 + s \cdot 1 = s \pmod{b}.$$

Set $x = x_0 \bmod ab$. The difference $x_0 - x$ is a multiple of ab, hence a multiple of both a and b. Thus reducing modulo ab preserves both congruences, and $x \in \mathbb{Z}_{ab}$ by the definition of the remainder.

(b) Uniqueness.

Let $t = x - y$. The two congruences imply $a \mid t$ and $b \mid t$. Multiplying the Bézout identity by t gives

$$t = aut + bvt.$$

Since $b \mid t$, the first term is divisible by ab; since $a \mid t$, the second term is too. Thus $ab \mid t$. If both $x, y \in \mathbb{Z}_{ab}$, then $-(ab-1) \le x - y \le ab - 1$, whose only multiple of ab is zero. Hence $x = y$.

(c) Applying the formula.

The given identity supplies $u = 3$ and $v = -2$ for $a = 5$, $b = 7$. For $r = 2$, $s = 3$, the formula gives

$$x = \big(2 \cdot 7 \cdot (-2) + 3 \cdot 5 \cdot 3\big) \bmod 35 = 17.$$

Indeed, $17 \equiv 2 \pmod{5}$ and $17 \equiv 3 \pmod{7}$.

(d) Why coprimality matters.

The system $x \equiv 0 \pmod{4}$, $x \equiv 1 \pmod{6}$ has no solution: the first condition makes x even, and the second makes x odd. Thus arbitrary remainders need not be compatible when the moduli share a factor.

Question 7 RSA works for every message

(a) Why Euler's theorem is not enough.

Euler's theorem requires $\gcd(m, N) = 1$. When $m = 5$ and $N = 35$, 5^{24} is divisible by 5, so it is 0 modulo 5. If it were 1 modulo 35, it would also be 1 modulo 5, a contradiction. This invalidates the proposed proof, not RSA decryption.

(b) Correctness modulo p.

If $p \mid m$, then $m^{ed} \equiv 0 \equiv m \pmod{p}$, since $ed \geq 1$. Otherwise Fermat's little theorem applies, and

$$\begin{aligned} m^{ed} &= m^{1+k(p-1)(q-1)} = m\big(m^{p-1}\big)^{k(q-1)} \\ &\equiv m \cdot 1^{k(q-1)} = m \pmod{p}. \end{aligned}$$

The integer $ed - 1 = k(p-1)(q-1)$ is a multiple of $p - 1$.

(c) Combining the two primes.

Interchanging p and q proves $m^{ed} \equiv m \pmod{q}$ as well: the divisible case gives zero, and the other case uses Fermat modulo q with exponent multiplier $k(p-1)$. The uniqueness argument in Question 6 now gives

$$m^{ed} \equiv m \pmod{pq}.$$

Because $c \equiv m^e \pmod{N}$, we have $c^d \equiv m^{ed} \equiv m \pmod{N}$. Both $c^d \bmod N$ and m lie in $\mathbb{Z}_N$, so they are equal. For $m = 0$, both prime-divisibility cases apply, and encryption and decryption both return zero. No inverse of m was used.

(d) A message sharing a factor with N.

Here $(p-1)(q-1) = 24$ and $ed = 25 = 1 + 24$, so $k = 1$. All the following congruences are modulo 35:

$$10^2 \equiv 30, \qquad 10^4 \equiv 25, \qquad c \equiv 10^5 \equiv 25 \cdot 10 \equiv 5.$$

Decrypt using

$$5^2 \equiv 25, \qquad 5^4 \equiv 30, \qquad 5^5 \equiv 30 \cdot 5 \equiv 10.$$

The recovered message is 10. Since $\gcd(10, 35) = 5$, the coprime-message proof from lecture does not apply to this example.

(e) Encryption is a bijection.

If $\mathcal{E}(m_1) = \mathcal{E}(m_2)$, apply the decryption function to both sides. Part (c) gives $m_1 = m_2$, so encryption is one-to-one. It maps the finite N-element set $\mathbb{Z}_N$ into itself; its N distinct outputs must therefore fill the codomain. Thus it is onto. Every ciphertext comes from exactly one message, and two different messages cannot collide, even if a message is not coprime to N.